Malware Analysis Practical 1

Step 1: Virus Scanning

All given files (Lab01-01.exe, Lab01-02.exe, Lab01-03.exe, Lab01-04.exe, Lab03 files) were uploaded to
VirusTotal.

- The files were scanned using multiple antivirus engines.
- Most of the files were detected as Trojan / Backdoor malware.
- Detection ratios confirmed that the files are malicious.

All samples matched existing antivirus signatures.

Step 2: Checking Compilation Details

The files were opened in
PEview.

- The TimeDateStamp field was examined.
- The files were compiled around 2010 (common in lab samples).

Step 3: Detecting Packing or Obfuscation

Each file was analyzed for packing using PEview / PEStudio.

- Some files (like Lab01-02.exe) showed:
    - Few imports
    - High entropy
    - Unusual sections

Indicating packed/obfuscated malware

- Other files showed:
    - Normal imports
    - Readable strings

Indicating not packed

Step 4: Import Table Analysis

The import table was analyzed to understand functionality.

Common imports found:

- kernel32.dll
  → File handling, process creation
- advapi32.dll
  → Registry modification

- ws2_32.dll
  → Network communication

Malware performs file operations, modifies registry, and communicates over network.

Step 5: String Analysis

Strings were extracted using
strings.

- Found:
  - File paths
  - Registry keys
  - IP addresses / URLs
  - Suspicious commands

Step 6: Resource Analysis

The file Lab01-04.exe was opened in
Resource Hacker.

- One resource was found in the resource section.
- The resource was extracted.
- It contained an embedded executable.

Malware hides payload inside resource section for stealth.

Step 7: Dynamic Analysis Execution

The malware samples (Lab03 files) were executed in a safe virtual environment.

- System behavior was monitored in real time.

Step 8: Process Monitoring

Using
Process Explorer:

- New suspicious processes were observed.
- Some malware injected itself into legitimate processes.
- High CPU or abnormal activity was noticed.

Step 9: System Activity Monitoring

Using
Process Monitor:

Filters applied:

- Process Name = malware
- Operations = CreateFile, RegSetValue

Observed:

- File creation
- Registry modification
- DLL loading

Step 10: Persistence Mechanism

- Malware created registry entries (Run keys)
- Ensured execution after system restart

Step 11: Network Analysis

Using Wireshark:

- Outgoing connections to suspicious IPs were observed
- Possible command-and-control (C2) communication detected

Network Indicators:

- IP addresses
- Domains
- Ports

Step 12: DLL Execution Analysis

For Lab03-02.dll:

- Executed using:

rundll32 Lab03-02.dll, function_name

- DLL installed itself and ran in background process
- Process location identified using Process Explorer

Step 13: Memory Behavior (Lab03-03.exe)

- Observed memory modification
- Code injection into other processes

Step 14: Anti-Analysis Behavior (Lab03-04.exe)

- Malware did not run normally
- Detected debugger or VM environment

Roadblock:

- Anti-debugging / anti-VM techniques

Bypass:

- Run in controlled or modified environment

Indicators Identified

Host-Based Indicators:

- File creation
- Registry changes
- Suspicious processes
- DLL injection

Network-Based Indicators:

- Suspicious IP addresses
- Domains
- Outgoing connections

Malware Analysis Practical 2

Step 1: Load the Malware File

1. Open IDA Pro
2. Click File → Open
3. Select file (e.g., Lab05-01.dll)
4. Choose:
    - ✓ Auto-analysis enabled
    - ✓ PE format

Wait until analysis completes

Step 2: Locate DllMain

1. Press Shift + F12 (Strings) or go to Functions window
2. Search for DllMain
3. Double-click it

Note the address (e.g., 0x1000D02E)

Step 3: Analyze Imports (gethostbyname)

1. Press Shift + F7 (Imports window)
2. Scroll to:
    - gethostbyname (inside ws2_32.dll)
3. Double-click it

IDA shows where it is used

Step 4: Find Cross-References

1. Press X on gethostbyname
2. Count number of references

This gives number of functions calling it

Step 5: Analyze DNS Request

1. Go to address 0x10001757
2. Look above the call instruction
3. Find:
    - push offset "domain_name"

That string = DNS request

Step 6: Check Local Variables & Parameters

1. Go to function 0x10001656
2. Press Tab (to switch to pseudocode if available)
3. Look at:
    - var_ → local variables
    - arg_ → parameters

Count them

Step 7: Locate "\cmd.exe /c"

1. Press Shift + F12 (Strings)
2. Search:

cmd.exe

3. Double-click string

IDA jumps to its location

Step 8: Analyze Command Execution

1. Check nearby instructions
2. Look for:
    - CreateProcess
    - WinExec
    - system

This shows malware is executing commands

Step 9: Analyze Global Variable

1. Go to:

dword_1008E5C4

2. Press X (cross-references)
3. See where it is modified

Understand how it controls flow

Step 10: Analyze memcmp Logic

1. Search:

memcmp

2. Check comparisons like:
    - "robotwork"

If match → specific command executed

Step 11: Analyze Export (PSLIST)

1. Go to:
    - Exports window
2. Find:
    - PSLIST
3. Open it

Check APIs:

- Process enumeration functions

Step 12: Graph Analysis

1. Open function sub_10004E79
2. Press Spacebar (Graph view)
3. Observe flow

Based on APIs:

- Rename function (press N)
  Example: CommandHandler

Step 13: Analyze API Calls

1. Open DllMain
2. Count:
    - Direct API calls
3. Follow calls (double-click)

Count depth 2 calls

Step 14: Analyze Sleep

1. Go to 0x10001358
2. Look above:

```
push value
call Sleep
```

Value = milliseconds

Step 15: Analyze socket()

1. Go to 0x10001701
2. Observe:

```
push protocol
push type
push domain
call socket
```

Step 16: Apply Symbolic Constants

1. Right-click parameter
2. Select Use standard enum

Convert to:

- AF_INET
- SOCK_STREAM
- IPPROTO_TCP

Step 17: Detect VMware

1. Search:

in

(opcode 0xED)

2. Look for:
    - "VMXh"

Confirms VM detection

Step 18: Check Encoded Data

1. Jump to:

0x1001D988

2. Observe raw data

Step 19: Run Python Script

1. Go to:
    - File → Script file
2. Run Lab05-01.py

It decodes hidden data

Step 20: Convert to ASCII

1. Select data
2. Press A

Converts to readable string

Lab06 (Quick Step Flow)

Step 21: Open Lab06 Files in IDA

- Load each file (Lab06-01 to 06-04)
- Let auto-analysis complete

Step 22: Find main()

1. Press G
2. Type:

main

Step 23: Analyze Code Flow

1. Observe:
    - Loops (while, for)
    - Conditions (if, switch)

Identify logic

Step 24: Analyze Subroutines

1. Double-click called functions
2. Identify:
    - String compare
    - Network calls
    - File operations

Step 25: Check Network Indicators

1. Open Strings (Shift + F12)
2. Look for:

- URLs
- IPs

Step 26: Determine Purpose

Based on:

- API calls
- Strings
- Flow

Identify:

Password checker , Data sender , Backdoor, Bot

Malware Analysis Practical 3

Step 1: Open Malware File

1. Open IDA Pro
2. File → Open → Select file (Lab07 / Lab09)
3. Let auto-analysis complete

Step 2: Find main() / Entry Point

1. Press G → type main or check entry point
2. Open the main function

This is where execution starts

Step 3: Check Persistence

1. Search for:
   - RegSetValue
   - CreateService
2. Open those functions

If found → malware adds itself to startup (registry)

Step 4: Check Mutex

1. Search (Shift + F12) → type:

mutex

2. Or look for:
   - CreateMutex

Used to prevent multiple copies running

Step 5: Find Strings

1. Press Shift + F12
2. Look for:
    - IP address
    - Domain
    - File paths

These give:

- Network indicators
- Host indicators

Step 6: Analyze Network Activity

1. Search for APIs:
    - socket
    - connect
    - send
2. Open them

Shows communication with attacker

Step 7: Identify Purpose

Check:

- Command execution (cmd.exe)
- Network calls
- File creation

Decide:

- Backdoor / Downloader / Bot

Step 8: Check Loop (Execution Time)

1. Look for:
    - while
    - jmp loops

If infinite loop → malware runs continuously

Lab09 (Dynamic Analysis using Debugger)

Step 9: Open in Debugger

1. Open OllyDbg
2. Load malware file

3. Press Run (F9)

Step 10: Check Command-Line Requirement

1. Look for:
    - strcmp
    - password comparison
2. Observe required input

Step 11: Patch Password Check

1. Find conditional jump (JZ / JNZ)
2. Modify instruction:
    - Change jump → NOP or unconditional jump

Removes password restriction

Step 12: Observe Behavior

1. Run program again
2. Check:
    - New process (CreateProcess)
    - File creation
    - Network activity

Step 13: Decode Hidden Data

1. Trace execution step-by-step (F7/F8)
2. Observe:
    - XOR / decoding loop
3. Find real domain/IP

Step 14: DLL Analysis (Lab09-03)

1. Load EXE in debugger
2. Check loaded DLLs
3. Note base addresses

Compare with IDA

Step 15: Track File Writing

1. Search:
    - WriteFile
2. Check filename parameter

Identifies dropped file

Step 16: Identify Indicators

Host-based:

- Registry keys
- Files created
- Mutex name

Network-based:

- IP address
- Domain
- Port

Malware Analysis Practical 4

Step 1: Setup

- Place driver (.sys) in:

C:\Windows\System32

- Keep executable file ready

Step 2: Open Monitoring Tool

- Open Process Monitor
- Apply filter:
    - Process Name = Lab10-01.exe

Step 3: Run Program

- Execute Lab10-01.exe
- Observe system activity

Step 4: Check Registry Activity

- Monitor registry operations in Process Monitor

Step 5: Kernel Debugging

- Open WinDbg
- Attach to system

Step 6: Set Breakpoint

- Set breakpoint on:

ControlService

Step 7: Execute and Trace

- Run program
- Trace execution into kernel

Step 8: Observe Driver Activity

- Monitor interaction with .sys file

Step 9: File Monitoring (Lab10-02.exe)

- Open Process Monitor
- Set filter: Lab10-02.exe
- Run program
- Observe file creation

Step 10: Check Kernel Usage

- Verify if any driver (.sys) is loaded

Step 11: Execute Lab10-03

- Place Lab10-03.sys in System32
- Run Lab10-03.exe

Step 12: Monitor System

- Use Process Monitor
- Observe service and driver loading

Step 13: Kernel Analysis

- Open WinDbg
- Trace kernel execution

Step 14: Stop Program

- Stop service or terminate process

Step 15: Observe Driver Behavior

- Monitor kernel-level activity

Malware Analysis Practical 5

a. Lab11-01.exe

Step 1:

- Open file in IDA Pro

Step 2:

- Press Shift + F12 → check strings
- Look for:
    - File names
    - Credentials-related strings

Step 3:

- Search for file APIs:

    - CreateFile
    - WriteFile
- Identify dropped file

Step 4:

- Search for registry APIs:
    - RegSetValue
- Check persistence mechanism

Step 5:

- Search for:
    - GetAsyncKeyState / keyboard APIs
- Identify credential stealing method

Step 6:

- Check network APIs:
    - send / connect
- Trace where credentials are sent

Step 7:

- Run program in test VM
- Enter username/password
- Monitor captured data

b. Lab11-02.dll

Step 8:

- Open DLL in IDA Pro

Step 9:

- Open Exports window
- Note exported functions

Step 10:

- Run using:

rundll32 Lab11-02.dll, <export>

Step 11:

- Check behavior after execution

Step 12:

- Locate Lab11-02.ini path using strings

Step 13:

- Search for registry/service APIs
- Identify persistence

Step 14:

- Search for API hooking:
    - SetWindowsHook
    - IAT modification

Step 15:

- Analyze hooked functions

Step 16:

- Identify target processes from code

Step 17:

- Open .ini file
- Check stored configuration/data

c. Lab11-03.exe + Lab11-03.dll

Step 18:

- Place both files in same folder

Step 19:

- Open EXE in IDA Pro

Step 20:

- Check strings for:
    - File names

- System file references

Step 21:

- Run EXE in VM
- Observe behavior

Step 22:

- Monitor file activity using: Process Monitor

Step 23:

- Identify DLL installation method

Step 24:

- Check which system file is modified

Step 25:

- Open DLL in IDA Pro
- Analyze functionality

Step 26:

- Search for file/storage APIs
- Identify where data is stored

Malware Analysis Practical 6

a. Lab12-01.exe + Lab12-01.dll

Step 1:

- Place both files in same folder
- Run Lab12-01.exe

Pop-up windows appear continuously

Step 2:

- Open Process Explorer
- Check process list

explorer.exe is injected

Step 3:

- Stop process using Process Explorer

Kill explorer.exe or malware process to stop pop-ups

Step 4:

- Open file in IDA Pro
- Search injection APIs

Malware injects DLL into process and shows pop-ups

b. Lab12-02.exe

Step 5:

- Open file in IDA Pro
- Analyze main function

Program acts as launcher

Step 6:

- Search for CreateProcess

Uses suspended process to hide execution

Step 7:

- Check resource/data section

Payload stored inside executable

Step 8:

- Analyze encoding

Payload is encrypted/encoded

Step 9:

- Check strings

Strings are obfuscated (encoded)

c. Lab12-03.exe

Step 10:

- Open in IDA Pro
- Analyze APIs

Acts as backdoor/injector

Step 11:

- Search injection APIs

Uses OpenProcess, WriteProcessMemory, CreateRemoteThread

Step 12:

- Monitor using Process Monitor

Creates files on disk

d. Lab12-04.exe

Step 13:

- Open in IDA Pro
- Go to 0x401000

Code prepares process injection

Step 14:

- Search injection APIs

explorer.exe is target process

Step 15:

- Find LoadLibraryA

Loads malicious DLL

Step 16:

- Find CreateRemoteThread

4th argument = address of LoadLibraryA

Lab12-01.dll is dropped by the main executable.

Malware Analysis Practical 7

a. Lab13-01.exe

Step 1: String Analysis

- Run:

strings Lab13-01.exe

- Observe readable vs missing strings
- Perform dynamic analysis (run in VM + monitor network)

Some network-related data and strings are not visible → encoded

Step 2: Search for XOR Encoding

- Open file in IDA Pro
- Press Shift + F12 → search "xor"
- Navigate to XOR loop

XOR-based encoding is used

Step 3: Identify XOR Key

- Trace XOR instruction:

xor byte ptr [eax], value

- Note constant value

Key = 0x6A, used to encode strings/data

Step 4: Use Static Plugins

- Run:
    - FindCrypt2
    - KANAL
    - Entropy Plugin

Base64 encoding also detected

Step 5: Network Encoding Analysis

- Capture traffic (Wireshark or similar)
- Observe outgoing data

Data is encoded using Base64

Step 6: Locate Base64 Function

- Search for Base64 table/string in IDA
- Trace encoding function

Base64 routine located in encoding subroutine

Step 7: Analyze Encoded Data

- Trace buffer before sending

Max length ≈ 64 bytes, encodes system/network info

Step 8: Check Padding

- Observe Base64 output

No "=" padding used

Step 9: Determine Behavior

- Analyze APIs (send, connect)

Malware encodes and sends data to remote server

b. Lab13-02.exe

Step 10: Dynamic Analysis

- Run in VM
- Monitor using Process Monitor

Creates encoded files on disk

Step 11: Static Encoding Check

- Open in IDA Pro
- Search:

xor

- Run FindCrypt2 / KANAL

XOR encoding detected

Step 12: Identify Relevant API

- Check imports:

- WriteFile

WriteFile used to store encoded content

Step 13: Locate Encoding Function

- Follow cross-references from WriteFile
- Trace backward

Encoding function located before file write

Step 14: Trace Data Source

- Follow buffer passed to encoding function

Original file/data is encoded

Step 15: Identify Algorithm

- Analyze loop logic

Simple XOR-based encoding

Step 16: Recover Original Data

- Use debugger or memory dump

Original data can be recovered before encoding

c. Lab13-03.exe

Step 17: Compare Strings vs Runtime

- Run strings
- Compare with dynamic behavior

Many strings are hidden/encoded

Step 18: Search XOR

- Open in IDA Pro
- Search for XOR loops

XOR encoding present

Step 19: Use Crypto Tools

- Run:
  - FindCrypt2
  - KANAL
  - Entropy Plugin

Cryptographic algorithm (RC4-like) detected

Step 20: Identify Encoding Types

- XOR encoding
- Cryptographic encryption

Step 21: Identify Keys

- Analyze XOR loop → key value
- Analyze crypto setup → key string
- XOR key = fixed byte
- Crypto key = hardcoded

Step 22: Analyze Encryption Requirement

Key alone not sufficient → algorithm required

Step 23: Determine Behavior

- Check network/file APIs

Encrypts data and sends to remote system

Step 24: Decrypt Data

- Extract encoded data
- Write simple XOR/crypto script

Decrypted output = readable data (system/network info)

Malware Analysis Practical 8

Step 1: Open File & Check Imports

- Open all samples in IDA Pro
- Check networking libraries

Uses WinINet / ws2_32 for HTTP communication

Step 2: String Analysis

- Press Shift + F12
- Identify:
    - URLs
    - IPs
    - Commands

Hardcoded URLs + encoded strings found

Step 3: Analyze Beacon Data

- Trace HTTP request building

Beacon contains system info + fixed strings, useful for attacker identification

Step 4: Encoding Detection

- Search xor + use FindCrypt2/KANAL

Uses XOR + modified Base64 (non-standard)

Step 5: Network Behavior

- Analyze send/connect APIs

Malware sends encoded beacon via HTTP GET

Step 6: Command Handling

- Trace response handling

Malware receives commands from server and executes them

Step 7: Signature Identification

- Identify fixed vs dynamic data
- Fixed → good signature (URL, patterns)
- Dynamic → avoid (system info)

Step 8: Anti-Analysis (Lab15-01)

- Analyze disassembly carefully

Uses anti-disassembly (junk/rogue opcodes)

- Hidden input required to print “Good Job!”

Step 9: Web Interaction (Lab15-02)

- Trace URL + HTTP request
- Requests webpage
- Extracts data
- Uses dynamic User-Agent

Step 10: Hidden Malware Behavior (Lab15-03)

- Trace hidden function calls
- Hidden malicious code executed
- Uses hardcoded URL + filename
- Performs background malicious actions

Malware Analysis Practical 9

Step 1: Load in Debugger

- Open samples in OllyDbg

Malware uses debugger detection techniques

Step 2: Identify Anti-Debugging (Lab16-01)

- Search for:
    - IsDebuggerPresent
    - CheckRemoteDebuggerPresent
    - PEB checks

Uses API checks + PEB flag checks

Step 3: Observe Behavior

- Run in debugger

Malware exits / changes behavior when debugger detected

Step 4: Bypass Anti-Debugging

- Patch:
    - Change JZ/JNZ
    - Modify return values

Bypass by patching or hiding debugger

Step 5: Modify Structures

- Edit memory in debugger (PEB → BeingDebugged = 0)

Manually disable debug flag

Step 6: Use Plugin

Use OllyDbgHide plugin to bypass detection

Step 7: Password Analysis (Lab16-02)

- Run from command line

Requires password argument

- Debug and trace strncmp

Password found in comparison

Step 8: Anti-Debugging in Lab16-02

- Check unusual PE structure + TLS callback

Uses TLS callback anti-debugging

- Disable check

Correct password visible in debugger

Step 9: Anti-Debugging + Domain (Lab16-03)

- Rename file if required

- Run and analyze
- Uses anti-debugging (API + timing checks)
- Exits if debugger found
- Domain identified in strings

Step 10: Anti-VM Analysis (Lab17)

- Run in VM

Malware detects VM and stops

- Check techniques:
    - VMware strings
    - Registry keys
    - I/O instructions

Uses VMware detection techniques

- Bypass:
    - Patch jumps
    - Remove VM artifacts

Malware runs after bypass

Malware Analysis Practical 10

Step 1:

- Run Lab19-01.bin using shellcode launcher tool to execute the shellcode in a safe environment.

The shellcode is XOR encoded to hide its content.

Step 2:

- Open the shellcode in IDA Pro and examine the disassembly carefully.

The shellcode uses manual import resolution instead of normal imports.

Step 3:

- Trace the code where API functions are resolved dynamically.

It manually loads APIs like LoadLibraryA and GetProcAddress.

Step 4:

- Monitor network activity using system tools while executing the shellcode.

The shellcode connects to a hardcoded remote host/IP address.

Step 5:

- Monitor file system changes using a tool like Process Monitor.

The shellcode creates files on disk (filesystem residue).

Step 6:

- Analyze overall behavior from execution and API calls.

The shellcode downloads and executes a malicious payload.

Step 7:

- Open Lab19-02.exe in a debugger and run it step-by-step.

The program injects shellcode into explorer.exe process.

Step 8:

- Locate where the shellcode is stored inside the executable.

The shellcode is stored in memory or embedded in the binary section.

Step 9:

- Analyze how the shellcode is protected before execution.

It is XOR encoded before being executed.

Step 10:

- Trace network communication after injection.

The injected shellcode communicates with a remote server.

Step 11:

- Open Lab19-03.pdf and analyze it or extract embedded shellcode.

The file uses a PDF exploit (Adobe vulnerability).

Step 12:

- Analyze extracted shellcode similarly to previous steps.

The shellcode is XOR encoded and hides malicious logic.

Step 13:

- Monitor system after execution of shellcode.

It drops files and executes malicious payload on the system.

Step 14:

- Open Lab20-01.exe in IDA Pro and analyze the main function.

It uses URLDownloadToFile to download a file from a URL.

Step 15:

- Trace the URL used in download function.

A hardcoded URL is used to fetch a file from the internet.

Step 16:

- Open Lab20-02.exe and analyze object creation and function calls.

The program uses C++ objects and virtual functions.

Step 17:

- Trace virtual function calls and network behavior.

The malware communicates with a server and executes commands.

Step 18:

- Open Lab20-03.exe and analyze control flow.

Uses exception handling (CxxThrowException) and switch-case logic.

Step 19:

- Run and analyze Lab21-01.exe with and without parameters.
- Requires command-line argument
- Uses strncmp to validate input
- Executes payload using CreateProcess

◆ Step 20:

- Analyze Lab21-02.exe on different architectures (x86 and x64).
- Detects system architecture (x86/x64)
- Drops different files based on system
- Executes different processes accordingly